



JAVASCRIPT MASTER STUDY DOCUMENTATION

CHAPTER 04: CONTROL FLOW & DECISION MAKING

TOTAL SECTIONS

45 Modules + Practice

CURRICULUM PART

Part 02 — Control Flow

CORE MECHANICS

Branching, Truthy & Short-Circuit

REFERENCE STANDARD

ECMAScript 2026 / V8 Runtime

Control Flow & Decision Making — Program কোন পথে চলবে তা নিয়ন্ত্রণ করা

JavaScript-এর code সাধারণত **উপরে থেকে নিচে, একটার পর একটা** execute হয়। কিন্তু বাস্তব program-এ সবসময় এমন হয় না। যেমন:

- বয়স 18 বা তার বেশি হলে → Account তৈরি করবে
- Password ভুল হলে → Error দেখাবে
- User logged in হলে → Dashboard দেখাবে
- Product stock-এ থাকলে → Buy button দেখাবে
- শুক্রবার হলে → বিশেষ message দেখাবে

এই ধরনের **condition** অনুযায়ী program-এর **execution path** পরিবর্তন করাকে বলা হয় **Control Flow**।

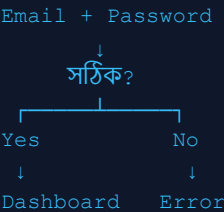
PART 01 — CORE BRANCHING & CONDITIONALS (4.1 – 4.10)

4.1 What is Control Flow?

Definition: Control Flow = Program-এর code কোন order-এ এবং কোন condition-এর ভিত্তিতে execute হবে।

REAL-LIFE EXAMPLE

ধরো তুমি একটি shopping website-এ login করছো। সেখানে সিদ্ধান্ত গ্রহণ পদ্ধতি:



JavaScript-এ এই decision নেওয়ার জন্য সবচেয়ে বেশি ব্যবহার হয়:

`if`, `else`, `else if`, `Nested if`, `switch`, `case`, `default`, `break`, `continue`, `Ternary operator`, `Truthy / Falsy`, `Logical conditions`

4.2 `if` Statement

`if` কী?

`if` ব্যবহার করে আমরা কোনো condition **true** হলে নির্দিষ্ট code execute করতে পারি।

SYNTAX

JAVASCRIPT SYNTAX

SYNTAX

```
if (condition) {  
    // code  
}
```

EXAMPLE

JAVASCRIPT

EXAMPLE

```
let age = 20;  
  
if (age ≥ 18) {  
    console.log("You are an adult.");  
}
```

OUTPUT

You are an adult.

কীভাবে কাজ করছে?

এখানে `age ≥ 18` অর্থাৎ `20 ≥ 18` এর ফলাফল `true`। তাই `{ }` এর ভিতরের code execute হয়েছে।

4.3 `if` Condition False হলে

JAVASCRIPT

FALSE CONDITION

```
let age = 15;  
  
if (age ≥ 18) {  
    console.log("You are an adult.");  
}
```

OUTPUT

কিছুই দেখাবে না

কারণ: `15 ≥ 18` এর ফলাফল `false`। তাই `if` block execute হয়নি।

4.4 `if...else`

যখন condition **true** হলে এক কাজ, আর **false** হলে অন্য কাজ করতে হবে, তখন `if ... else` ব্যবহার করি।

SYNTAX

JAVASCRIPT SYNTAX

SYNTAX

```
if (condition) {  
  // true হলে  
} else {  
  // false হলে  
}
```

EXAMPLE

JAVASCRIPT

VOTING CHECK

```
let age = 16;  
  
if (age ≥ 18) {  
  console.log("You can vote.");  
} else {  
  console.log("You cannot vote yet.");  
}
```

OUTPUT

You cannot vote yet.

REAL-LIFE EXAMPLE — ATM-এ

```
Balance ≥ Withdraw Amount?  
↓  
Yes → টাকা দাও  
No → Insufficient Balance
```

JAVASCRIPT

ATM FLOW

```
let balance = 5000;  
let withdraw = 7000;  
  
if (balance ≥ withdraw) {  
  console.log("Please collect your money.");  
} else {  
  console.log("Insufficient balance.");  
}
```

OUTPUT

Insufficient balance.

4.5 `if...else if...else`

যখন একাধিক condition check করতে হবে তখন `else if` ব্যবহার করি।

SYNTAX

JAVASCRIPT SYNTAX

SYNTAX

```
if (condition1) {  
  // code  
} else if (condition2) {  
  // code  
} else if (condition3) {  
  // code  
} else {  
  // code  
}
```

EXAMPLE — MARKS

JAVASCRIPT

GRADE CALCULATION

```
let marks = 85;  
  
if (marks ≥ 80) {  
  console.log("Grade A+");  
} else if (marks ≥ 70) {  
  console.log("Grade A");  
} else if (marks ≥ 60) {  
  console.log("Grade A-");  
} else if (marks ≥ 50) {  
  console.log("Grade B");  
} else {  
  console.log("Fail");  
}
```

OUTPUT

Grade A+

Important: JavaScript উপরের condition থেকে check করে। একটি matching condition পাওয়ার পর নিচেরগুলো আর check করে না।

4.6 Multiple Conditions (`&&` — AND)

একটি `if` -এর মধ্যে একাধিক condition দেওয়া যায়। এখানে আমরা Chapter 3-এর logical operators ব্যবহার করব।

`&&` — AND: দুটো condition-ই true হতে হবে।

JAVASCRIPT

LOGICAL AND

```
let age = 25;  
let hasNID = true;  
  
if (age ≥ 18 && hasNID) {  
  console.log("You can open the account.");  
}
```

OUTPUT

You can open the account.

কারণ: `age ≥ 18` → `true` এবং `hasNID` → `true`; সুতরাং `true && true` → `true`।

4.7 `||` – OR

`||` – OR: যেকোনো একটি condition true হলেই হবে।

JAVASCRIPT

LOGICAL OR

```
let hasEmail = true;
let hasPhone = false;

if (hasEmail || hasPhone) {
  console.log("You can create an account.");
}
```

OUTPUT

You can create an account.

কারণ: `true || false` → `true`।

4.8 `!` – NOT

! condition-এর result উল্টে দেয়।

JAVASCRIPT

LOGICAL NOT

```
let isLoggedIn = false;

if (!isLoggedIn) {
  console.log("Please login first.");
}
```

OUTPUT

Please login first.

কারণ: `isLoggedIn = false`, তাই `!false` হয়ে যায় `true`।

4.9 Nested `if`

একটি `if` -এর ভিতরে আরেকটি `if` থাকলে তাকে **Nested if** বলে।

JAVASCRIPT

NESTED IF

```
let age = 22;
let hasNID = true;

if (age ≥ 18) {
  if (hasNID) {
    console.log("Account can be created.");
  }
}
```

OUTPUT

Account can be created.

REAL-LIFE EXAMPLE — BANK ACCOUNT খুলতে

```
Age >= 18?
↓ Yes
NID আছে?
↓ Yes
Account Create
```

JAVASCRIPT

BANK ACCOUNT LOGIC

```
if (age ≥ 18) {
  if (hasNID) {
    console.log("Account created.");
  }
}
```

4.10 Nested `if...else`

JAVASCRIPT

NESTED IF...ELSE

```
let isLoggedIn = true;
let isAdmin = false;

if (isLoggedIn) {
  if (isAdmin) {
    console.log("Welcome Admin.");
  } else {
    console.log("Welcome User.");
  }
} else {
  console.log("Please login.");
}
```

OUTPUT

Welcome User.

4.11 `switch` Statement

যখন একটি variable-এর একাধিক নির্দিষ্ট value check করতে হয়, তখন `switch` অনেক usefull

SYNTAX

JAVASCRIPT SYNTAX

SYNTAX

```
switch (value) {  
  case value1:  
    // code  
    break;  
  case value2:  
    // code  
    break;  
  default:  
    // code  
}
```

EXAMPLE

JAVASCRIPT

DAY OF WEEK

```
let day = 3;  
  
switch (day) {  
  case 1:  
    console.log("Saturday");  
    break;  
  case 2:  
    console.log("Sunday");  
    break;  
  case 3:  
    console.log("Monday");  
    break;  
  case 4:  
    console.log("Tuesday");  
    break;  
  default:  
    console.log("Invalid day");  
}
```

OUTPUT

Monday

কারণ: `day = 3` তাই `case 3` match করেছে।

4.12 `case`

`switch` -এর মধ্যে প্রতিটি possible value-কে `case` বলা হয়।

JAVASCRIPT

CASE STRUCTURE

```
switch (day) {  
  case 1:  
    console.log("Saturday");  
    break;  
  case 2:  
    console.log("Sunday");  
    break;  
}
```

এখানে `case 1` এবং `case 2` দুটি possible case।

4.13 `break`

`break` ব্যবহার করে `switch` execution বন্ধ করা হয়।

JAVASCRIPT

BREAK IN ACTION

```
let number = 2;  
  
switch (number) {  
  case 1:  
    console.log("One");  
    break;  
  case 2:  
    console.log("Two");  
    break;  
  case 3:  
    console.log("Three");  
    break;  
}
```

OUTPUT

Two

`case 2` match করার পর `break;` এর কারণে switch শেষ হয়ে যায়।

4.14 `break` না দিলে কী হয়? (Fall-through)

খুব গুরুত্বপূর্ণ: `break` না দিলে নিচের সব case execute হয়ে যায়, একে **Fall-through** বলে!

JAVASCRIPT

FALL-THROUGH BUG

```
let number = 2;

switch (number) {
  case 1:
    console.log("One");
  case 2:
    console.log("Two");
  case 3:
    console.log("Three");
  default:
    console.log("Other");
}
```

OUTPUT

```
Two
Three
Other
```

ব্যাখ্যা: অর্থাৎ `case 2` match করার পর `break` না থাকায় পরের case-গুলোর code-ও execute করেছে।

4.15 Intentional Fall-through

কখনো কখনো আমরা ইচ্ছা করেই একাধিক case একই code-এর জন্য ব্যবহার করি।

JAVASCRIPT

GROUPED CASES

```
let day = "Saturday";

switch (day) {
  case "Saturday":
  case "Sunday":
    console.log("Weekend");
    break;
  default:
    console.log("Working day");
}
```

OUTPUT

```
Weekend
```

এখানে Saturday এবং Sunday দুটোতেই একই output প্রদর্শন করছে।

4.16 `default`

কোনো `case` match না করলে `default` execute হয়।

JAVASCRIPT

DEFAULT CASE

```
let day = 10;

switch (day) {
  case 1:
    console.log("Saturday");
    break;
  case 2:
    console.log("Sunday");
    break;
  default:
    console.log("Invalid day");
}
```

OUTPUT

Invalid day

REAL-LIFE EXAMPLE — PAYMENT METHOD

```
Payment Method:
1 → bKash
2 → Nagad
3 → Card
অন্য কিছু → Invalid Payment Method
```

4.17 `switch` বনাম `if...else`

দুটো দিয়েই decision নেওয়া যায়। কিন্তু ব্যবহারের ক্ষেত্র ভিন্ন:

`if...else`: Range বা complex condition-এর জন্য ভালো:

JAVASCRIPT

RANGE CHECK

```
let marks = 75;
if (marks ≥ 80) {
  console.log("A+");
} else if (marks ≥ 70) {
  console.log("A");
}
```

`switch`: Specific value-এর জন্য convenient:

JAVASCRIPT

EXACT MATCH

```
let role = "admin";
switch (role) {
  case "admin":
    console.log("Admin Panel");
    break;
  case "user":
    console.log("User Dashboard");
    break;
}
```

তুলনামূলক চার্ট

Situation	সাধারণত ব্যবহার্য
Range check	<code>if ... else</code>
Complex condition	<code>if ... else</code>
Exact value match	<code>switch</code>
অনেকগুলো নির্দিষ্ট option	<code>switch</code>

PART 03 — TRUTHY, FALSY & MODERN SHORT-CIRCUITING (4.18 – 4.31)

4.18 Truthy & Falsy

JavaScript-এর অন্যতম গুরুত্বপূর্ণ concept। `if` condition-এর মধ্যে JavaScript কোনো value-কে automatically **true** বা **false** হিসেবে evaluate করে।

Value in Condition

```
↓
├── Truthy
│   (True মত)
└── Falsy
    (False মত)
```

4.19 Falsy Values

JavaScript-এ প্রধান ৮টি falsy values:

false, 0, -0, 0n, "" (empty string), null, undefined, NaN

এগুলো condition-এর মধ্যে false হিসেবে behave করে।

JAVASCRIPT

FALSY EVALUATION

```
let username = "";

if (username) {
  console.log("Username exists.");
} else {
  console.log("Username is empty.");
}
```

OUTPUT

Username is empty.

কারণ: "" একটি falsy value।

4.20 Truthy Values

Falsy ছাড়া প্রায় সব value truthy। উদাহরণ: "hello", 42, -10, [], {}, true।

JAVASCRIPT

TRUTHY EVALUATION

```
let username = "Shariar";

if (username) {
  console.log("Username exists.");
}
```

OUTPUT

Username exists.

4.21 Empty Array `[]` কি falsy?

উত্তর: না। এটি truthy!

JAVASCRIPT

EMPTY ARRAY

```
if ([]) {
  console.log("Array is truthy");
}
```

OUTPUT

Array is truthy

4.22 Empty Object `{}` কি falsy?

উত্তর: না! Empty Object-ও **truthy**! এটা beginnerদের জন্য খুব important!

JAVASCRIPT

EMPTY OBJECT

```
if ({}) {  
  console.log("Object is truthy");  
}
```

OUTPUT

Object is truthy

4.23 Conditional Operator / Ternary Operator

আগের Chapter-এ আমরা Ternary দেখেছি। এটা `if ... else` -এর ছোট version।

SYNTAX

JAVASCRIPT SYNTAX

SYNTAX

```
condition ? valueIfTrue : valueIfFalse;
```

EXAMPLE

JAVASCRIPT

TERNARY

```
let age = 20;  
let result = age ≥ 18 ? "Adult" : "Minor";  
console.log(result);
```

OUTPUT

Adult

Equivalent `if...else`:

JAVASCRIPT

EQUIVALENT CODE

```
let age = 20;  
let result;  
if (age ≥ 18) {  
  result = "Adult";  
} else {  
  result = "Minor";  
}  
console.log(result);
```

দুটোর output ভুব্ একই।

4.24 Real-Life Ternary Example

ধরো website-এ login status দেখাতে হবে:

JAVASCRIPT

LOGIN STATUS

```
let isLoggedIn = true;

let message = isLoggedIn
  ? "Welcome back!"
  : "Please login.";

console.log(message);
```

OUTPUT

Welcome back!

4.25 Nested Ternary

Ternary-এর ভিতরে Ternary দেওয়া যায়।

JAVASCRIPT

NESTED TERNARY

```
let marks = 85;

let grade =
  marks ≥ 80 ? "A+"
  : marks ≥ 70 ? "A"
  : marks ≥ 60 ? "A-"
  : "Fail";

console.log(grade);
```

OUTPUT

A+

কিন্তু সাবধান ⚠️: অনেক বেশি nested ternary code difficult to read করতে পারে। তাই complex logic হলে `if ... else` ব্যবহার করা বেশি readable।

4.26 Short-Circuit Decision Making

Chapter 3-এ আমরা short-circuit দেখেছি। এখন control flow-এর সঙ্গে এটি ব্যবহার করি।

‘&&’ শর্ট-সার্কিট গার্ড প্যাটার্ন

JAVASCRIPT

SHORT-CIRCUIT GUARD

```
let isLoggedIn = true;

isLoggedIn && console.log("Show Dashboard");
```

OUTPUT

Show Dashboard

যদি `isLoggedIn = false;` হতো, তবে কিছু execute হতো না। কারণ `false && anything → false`।

4.27 `||` দিয়ে Default Value

JAVASCRIPT

DEFAULT FALLBACK

```
let username = "";

let displayName = username || "Guest";

console.log(displayName);
```

OUTPUT

Guest

কারণ: "" || "Guest" → "Guest" | Website-এ username না থাকলে Guest দেখাবে।

4.28 `??` Nullish Coalescing

|| এবং ?? এক জিনিস না। ?? শুধুমাত্র null অথবা undefined হলে right side value নেয়।

JAVASCRIPT

NULLISH FALLBACK

```
let username = null;
let name = username ?? "Guest";
console.log(name); // Guest

let username2 = "";
let name2 = username2 ?? "Guest";
console.log(name2); // "" (Empty string সংরক্ষিত থাকবে কারণ এটি null বা undefined নয়)
```

OUTPUT

Guest
"" (খালি স্ট্রিং)

4.29 `||` বনাম `??`

JAVASCRIPT

COMPARISON

```
let value = 0;

console.log(value || 100);
console.log(value ?? 100);
```

OUTPUT

100
0

সহজে মনে রাখবে:

- || → falsy (যেমন: 0, "") হলেই fallback ড্যাঁলু নেয়।
- ?? → শুধুমাত্র null / undefined হলেই fallback ড্যাঁলু নেয় (0 বা "" বহাল রাখে)।

4.30 Optional Chaining `?.` দিয়ে Safe Decision

ধরো `let user = {};`। এখন `console.log(user.profile.name);` দিলে `TypeError` দিয়ে crash করতে পারে কারণ `profile` নেই।

Optional chaining দিলে crash না করে `undefined` রিটার্ন করে:

JAVASCRIPT

OPTIONAL CHAINING

```
let user = {};  
console.log(user.profile?.name);
```

OUTPUT

```
undefined
```

এতে program crash না করে নিরাপদে `undefined` পাওয়া যায়।

4.31 `if` + Optional Chaining

JAVASCRIPT

SAFE CONDITION

```
let user = {  
  profile: {  
    name: "Sharir"  
  }  
};  
  
if (user.profile?.name) {  
  console.log("Name:", user.profile.name);  
}
```

OUTPUT

```
Name: Sharir
```

4.32 Multiple Conditions Example — Login System

এবার একটা real-world login system দেখি:

JAVASCRIPT

AUTH VERIFICATION

```
let email = "user@gmail.com";
let password = "123456";
let isVerified = true;

if (
  email === "user@gmail.com" &&
  password === "123456" &&
  isVerified
) {
  console.log("Login successful.");
} else {
  console.log("Invalid login information.");
}
```

OUTPUT

Login successful.

এখানে ৩টি condition: `email === "user@gmail.com"`, `password === "123456"`, এবং `isVerified`। সবকটি true হওয়ায় `true && true && true → true`।

4.33 E-commerce Example

ধরো একটি পণ্য ক্রয়ের লজিক:

JAVASCRIPT

PURCHASE DECISION

```
let price = 1500;
let stock = 5;
let isLoggedIn = true;

if (isLoggedIn && stock > 0) {
  console.log("You can buy this product.");
} else {
  console.log("You cannot buy this product.");
}
```

OUTPUT

You can buy this product.

4.34 Discount System

JAVASCRIPT

TIERED DISCOUNT

```
let totalAmount = 6000;

if (totalAmount ≥ 10000) {
  console.log("20% discount");
} else if (totalAmount ≥ 5000) {
  console.log("10% discount");
} else if (totalAmount ≥ 2000) {
  console.log("5% discount");
} else {
  console.log("No discount");
}
```

OUTPUT

10% discount

REAL-LIFE FLOW

```
ট10,000+ → 20%
ট5,000+ → 10%
ট2,000+ → 5%
এর কম → No discount
```

4.35 Grading System

এটা নিজে খুব ভালোভাবে practice করবে। পুরো গ্রেডিং ল্যাডার:

JAVASCRIPT

GRADE LADDER

```
let marks = 76;

if (marks ≥ 80) {
  console.log("A+");
} else if (marks ≥ 70) {
  console.log("A");
} else if (marks ≥ 60) {
  console.log("A-");
} else if (marks ≥ 50) {
  console.log("B");
} else if (marks ≥ 40) {
  console.log("C");
} else if (marks ≥ 33) {
  console.log("D");
} else {
  console.log("F");
}
```

OUTPUT

A

4.36 Login + Role Based Access (RBAC)

এটা real-world web development-এ খুব common:

JAVASCRIPT

RBAC LOGIC

```
let isLoggedIn = true;
let role = "admin";

if (isLoggedIn) {
  if (role === "admin") {
    console.log("Welcome to Admin Dashboard.");
  } else if (role === "teacher") {
    console.log("Welcome to Teacher Dashboard.");
  } else {
    console.log("Welcome to User Dashboard.");
  }
} else {
  console.log("Please login first.");
}
```

OUTPUT

Welcome to Admin Dashboard.

এই ধরনের logic পরে React, Node.js, Express, authentication system ইত্যাদিতে অনেক কাজে লাগবে।

4.37 `switch` দিয়ে Role System

একই কাজ `switch` দিয়েও পরিস্কারভাবে করা যায়:

JAVASCRIPT

SWITCH RBAC

```
let role = "teacher";

switch (role) {
  case "admin":
    console.log("Admin Dashboard");
    break;
  case "teacher":
    console.log("Teacher Dashboard");
    break;
  case "student":
    console.log("Student Dashboard");
    break;
  default:
    console.log("Unknown Role");
}
```

OUTPUT

Teacher Dashboard

4.38 Important Difference: `==` vs `===`

Control flow-এ এটি অনেক important:

JAVASCRIPT

LOOSE EQUALITY TRAP

```
let age = "18";

if (age == 18) {
  console.log("Matched"); // Type coercion হয়, তাই execute করে
}

if (age === 18) {
  console.log("Matched"); // Type ভিন্ন ("18" string vs 18 number), তাই কিছুই দেখাবে না
}
```

Best Practice: সাধারণত condition-এ সবসময় strict equality `===` ব্যবহার করা উচিত।**4.39 Common Mistake: `=` এবং `===`****ভুল (Assignment Bug):**

```
if (age = 18) { console.log("Adult"); }
```

এখানে `=` হলো assignment operator। এটি variable-এ 18 assign করে দেয় এবং 18 truthy হওয়ায় সর্বদা true হয়ে যায়।**সঠিক (Comparison):**

```
if (age === 18) { console.log("Adult"); }
```

4.40 Common Mistake: Semicolon নিয়ে সমস্যা**ভুল Logic:**

```
if (age ≥ 18); { console.log("Adult"); }
```

এখানে `;` এর কারণে `if` -এর স্টেটমেন্ট সেখানেই শেষ হয়ে যায় এবং `{ }` ব্লকটি সাধারণ ব্লক হিসেবে সবসময় রান হয়।**সঠিক:** `if (age ≥ 18) { console.log("Adult"); }` (if ব্র্যাকেটের শেষে সেমিকোলন দেওয়া যাবে না)।

4.41 Curly Braces ব্যবহার করা

Technically single statement হলে braces ছাড়া লেখা যায়:

JAVASCRIPT

SINGLE STATEMENT

```
if (age ≥ 18)
  console.log("Adult");
```

কিন্তু professional code-এ সবসময় কার্লি ব্রেসেস { } ব্যবহার করা হয়:

JAVASCRIPT

BEST PRACTICE

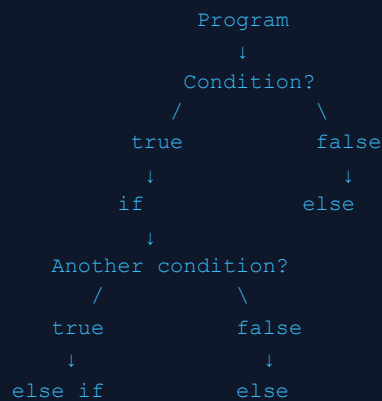
```
if (age ≥ 18) {
  console.log("Adult");
}
```

এতে code readable এবং maintainable হয়।

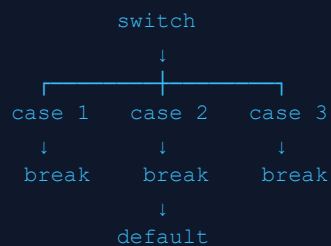
4.42 Control Flow-এর Complete Picture

এখন পুরো Chapter-এর concept একসাথে:

IF - ELSE IF - ELSE FLOW



SWITCH - CASE FLOW



4.43 Real-World Mini Project — ATM Decision System

এবার সব concept ব্যবহার করে একটি বাস্তবমুখী ATM প্রজেক্টের লজিক:

JAVASCRIPT

ATM ENGINE

```
let balance = 10000;
let withdrawAmount = 3000;
let pinCorrect = true;

if (pinCorrect) {
  if (withdrawAmount ≤ balance) {
    balance -= withdrawAmount;
    console.log("Withdrawal successful.");
    console.log("Withdrawn:", withdrawAmount);
    console.log("Remaining balance:", balance);
  } else {
    console.log("Insufficient balance.");
  }
} else {
  console.log("Incorrect PIN.");
}
```

OUTPUT

```
Withdrawal successful.
Withdrawn: 3000
Remaining balance: 7000
```

কী হচ্ছে? প্রথমে `pinCorrect` চেক করছে → সঠিক হলে `withdrawAmount ≤ balance` চেক করছে → টাকা কেটে নিয়ে অবশিষ্ট ব্যালেন্স `10000 - 3000 = 7000` আপডেট করছে।

4.44 Mini Project — Online Shopping Checkout

JAVASCRIPT

CHECKOUT FLOW

```
let isLoggedIn = true;
let productInStock = true;
let price = 2500;
let balance = 3000;

if (isLoggedIn) {
  if (productInStock) {
    if (balance ≥ price) {
      balance -= price;
      console.log("Order placed successfully.");
      console.log("Remaining balance:", balance);
    } else {
      console.log("Insufficient balance.");
    }
  } else {
    console.log("Product is out of stock.");
  }
} else {
  console.log("Please login first.");
}
```

OUTPUT

```
Order placed successfully.
Remaining balance: 500
```

4.45 Chapter 4 — Quick Cheat Sheet

Concept	কাজ
<code>if</code>	Condition true হলে code চালায়
<code>else</code>	Condition false হলে code চালায়
<code>else if</code>	Multiple condition check করে
Nested <code>if</code>	<code>if</code> -এর ভিতরে <code>if</code>
<code>switch</code>	Specific value match করে
<code>case</code>	<code>switch</code> -এর possible value
<code>break</code>	<code>switch</code> /loop থেকে বের হয়
<code>default</code>	কোনো case match না করলে চলে
<code>&&</code>	সব condition true হতে হবে
<code> </code>	যেকোনো একটি true হলেই হবে
<code>!</code>	Boolean result উল্টায়
<code>?:</code>	Short <code>if ... else</code> (Ternary)
<code>??</code>	<code>null</code> / <code>undefined</code> হলে fallback
<code>?.</code>	Safe property access (Optional Chaining)
Truthy	<code>if</code> -এ true হিসেবে behave করে
Falsy	<code>if</code> -এ false হিসেবে behave করে

Chapter 4 — Must Remember

1 ``if``:

```
if (condition) {  
  // code  
}
```

2 ``if...else``:

```
if (condition) {  
  // true  
} else {  
  // false  
}
```

3 ``else if``:

```
if (c1) { }  
else if (c2) { }  
else { }
```

4 ``switch``:

```
switch (val) {  
  case 1: break;  
  default:  
}
```

5 Ternary:

```
condition ? trueVal : falseVal;
```

6 Truthy/Falsy:

```
if (value) { // truthy }
```

7 Default value (`??`):

```
let name = username ?? "Guest";
```

Chapter 4 — Practice Questions

BASIC CONCEPTUAL QUESTIONS

1. `if` statement কী?
2. `if` এবং `else` -এর মধ্যে পার্থক্য কী?
3. `else if` কখন ব্যবহার করা হয়?
4. `switch` কী?
5. `break` কেন ব্যবহার করা হয়?
6. `default` কী?
7. Truthy এবং Falsy কী?
8. JavaScript-এর প্রধান falsy values কী কী?
9. `&&` এবং `||` -এর মধ্যে পার্থক্য কী?
10. Ternary operator কী?

CODING PRACTICE PROBLEMS

Problem 1 — Even/Odd:

```
let number = 25;
```

Check করো number even নাকি odd।

Expected: `Odd`

Problem 2 — Age Checker:

```
let age = 17;
```

Output দেখাও: `Adult` অথবা `Minor`।

Problem 3 — Grade System:

```
let marks = 92;
```

গ্রেডিং স্কেল অনুযায়ী Grade বের করো।

Problem 4 — Login System:

```
let email = "admin@gmail.com";  
let password = "1234";  
let isVerified = true;
```

সব ঠিক থাকলে: `Login Successful`, না হলে: `Login Failed`।

Problem 5 — Shopping:

```
let price = 5000;  
let balance = 6000;  
let inStock = true;
```

Check করো product কেনা যাবে কি না।

Chapter 4 শেষ করার পর তুমি যা বুঝবে

এই Chapter শেষ করলে তুমি JavaScript-এ:

```
Condition
  ↓
Decision
  ↓
Different Code Path
```

এই পুরো concept বুঝে যাবে। এগুলোই পরে বাস্তব application-এর:

- Login system
- Signup validation
- Admin/User role
- Shopping cart
- Payment checking
- Attendance system
- Dashboard access
- Form validation
- API response handling

ইত্যাদির basic decision-making logic তৈরি করবে।

Next Chapter: Chapter 5 — Loops & Iteration

পরবর্তী অধ্যায়ে আমরা একদম বিস্তারিতভাবে শিখব:

`for` loop, `while` loop, `do ... while`, `for ... of`, `for ... in`, Nested loops, `break`, `continue`, Infinite loop, Loop control, Arrays-এর সঙ্গে loop, Objects-এর সঙ্গে loop, Real-life examples, Pattern printing, Practical mini projects।

এবং প্রতিটি topic হবে Concept → Code → Output → সহজ Bangla explanation → Real-life example এই একই format-এ।